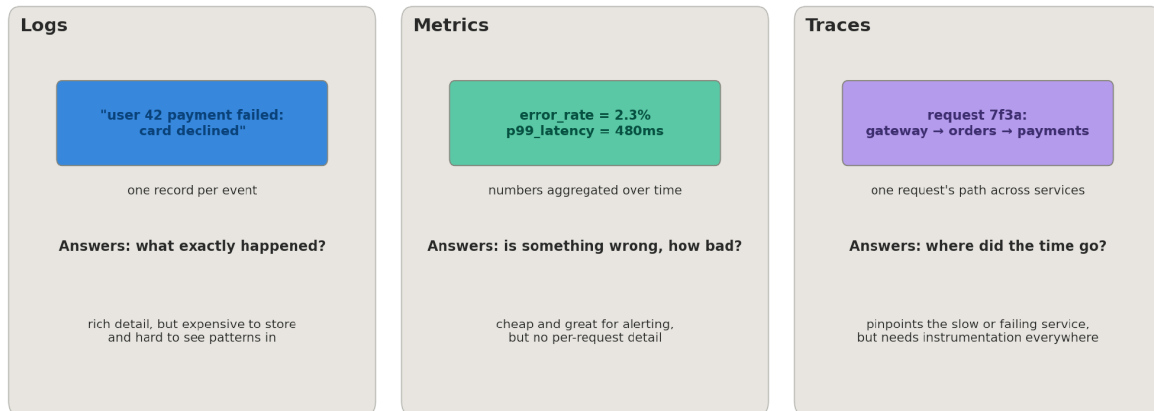


Stage 7 Crash Course: Observability

A single machine barely needs monitoring. Once you have many services and many nodes (Stages 2–6), you can no longer understand the system by looking at one box — observability is how you answer “is it broken, and where” without guessing.

1. The Three Pillars



They are complementary, not interchangeable: metrics tell you something is wrong, traces tell you where, and logs tell you why. A typical investigation moves through them in that order.

2. Logging

- **Structured logs**: emit logs as key-value records (JSON) rather than free-form sentences, so they can be filtered and aggregated by field (`user_id`, `request_id`, `status`) instead of regex-searched.
- **Log levels**: DEBUG, INFO, WARN, ERROR — so verbosity can be dialed up or down without code changes. Production usually runs at INFO or above; DEBUG volume is expensive.
- **Aggregation**: with many servers, logs must be shipped to a central searchable store (e.g., an ELK stack or a hosted equivalent). Logs sitting on individual machines are effectively unreachable during an incident, especially if that machine is the one that died.
- **Correlation IDs**: attach one request ID to every log line produced while handling a request, across every service — this is what lets you reassemble a single request's story from thousands of interleaved lines.

The cost side: logs are the most detailed pillar and also the most expensive to store and query at scale. Logging everything at high volume is a real cost problem, not just a style choice.

3. Metrics

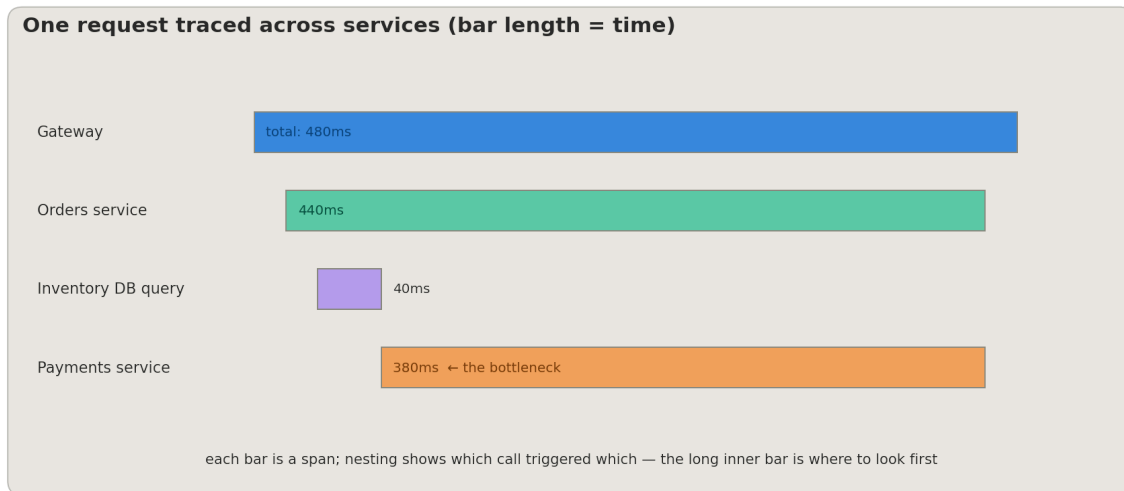
Metrics are numeric measurements aggregated over time — cheap to store, fast to query, and the right basis for dashboards and alerts.

Method	What it measures	Best applied to
RED	Rate (requests/sec), Errors (failed requests), Duration (latency distribution)	Request-driven services: APIs, microservices
USE	Utilization (how busy), Saturation (how much is queued/waiting), Errors	Resources: CPUs, disks, network links, queues

- Use percentiles, not averages, for latency. An average of 100ms can hide a p99 of 5 seconds — and the p99 is what your unluckiest users (often your heaviest ones) actually experience. This ties directly back to Stage 1's point that latency rises sharply near saturation, which averages smooth over.
- Cardinality: every unique combination of label values (e.g., a metric labeled by user_id) creates a separate time series. Labeling a metric with a high-cardinality field like user ID can create millions of series and blow up storage and cost. Labels should be low-cardinality (endpoint, status code, region); per-user detail belongs in logs or traces.

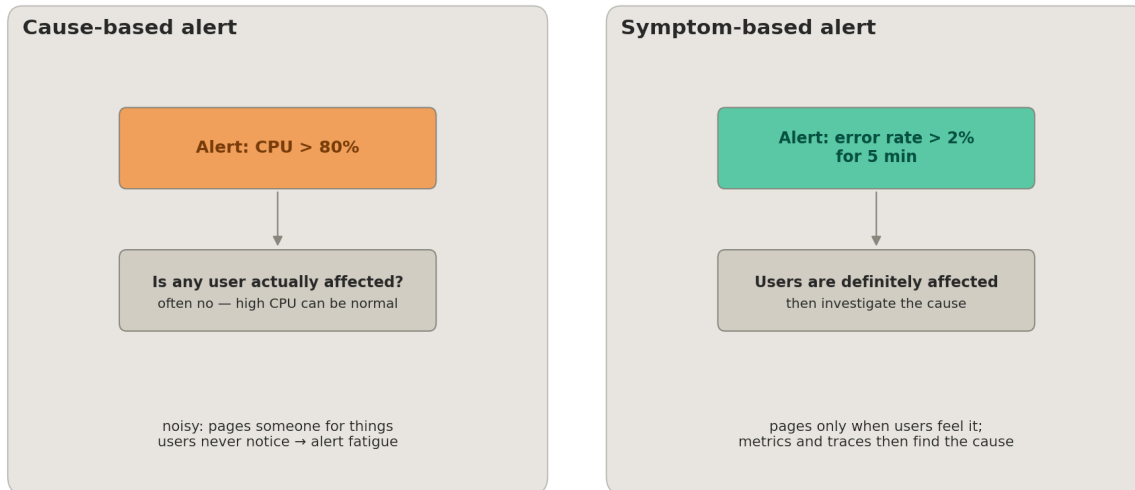
4. Distributed Tracing

A trace follows a single request through every service it touches. Each unit of work is a span (with a start time and duration); spans nest to show which call triggered which.



- Context propagation: the trace ID has to be passed along with every downstream call (typically as an HTTP header) so each service attaches its spans to the same trace. If one service in the chain fails to forward it, the trace breaks into disconnected pieces.
- What it buys you: in the example above, logs from each service would each look reasonable on their own, and metrics would show the gateway is slow. Only the trace shows that the time is going to the payments call rather than the database.
- Sampling: tracing every request at high traffic is expensive, so systems typically sample (e.g., 1% of requests, plus all errors and unusually slow requests).

5. Alerting



- Symptom-based alerts fire on what users experience (error rate, latency, availability). They page rarely and meaningfully — if one fires, someone is being hurt right now.
- Cause-based alerts fire on internal conditions (CPU, memory, queue depth). They're useful as dashboards and for diagnosis, but as pages they're noisy, since many internal anomalies never touch a user.

Alert fatigue: if on-call engineers get paged for things that don't matter, they learn to ignore pages — including the one that does. Every page should be actionable and urgent; anything else belongs on a dashboard or in a ticket, not a pager.

Tie to reliability targets: alerting thresholds should connect back to Stage 1's availability goals. If the target is 99.9%, the error-rate and latency alerts should fire well before that error budget is exhausted, not after.

How this stage connects back

- Percentile latency metrics are Stage 1's latency/throughput curve made measurable — the sharp rise near saturation shows up in p99 long before it shows up in the average.
- Distributed tracing exists because of Stage 6's microservices decision: once a request crosses many services, no single service's logs tell the whole story.
- ML-serving observability (Stage 8) is this same stack applied to inference traffic, with model-specific signals (prediction drift, feature distributions) added on top.

Must be able to explain before moving on

1. What question each of logs, metrics, and traces answers best, and in what order you'd use them during an incident.
2. Why average latency is a misleading metric and what to use instead.
3. Why labeling a metric with `user_id` is a problem (cardinality), and where per-user detail should go.
4. Why a CPU > 80% page is a worse primary alert than an elevated error rate, and what alert fatigue costs a team.